

Introduction to Reinforcement Learning

Applied Deep Learning — Chapter 14

Yin Yang

Foundation Books Project

Roadmap

RL as Learning from Interaction

Problem Setting and Formalism

Policies, Baselines, Evaluation

Values, Q-Functions, Credit Assignment

Q-Learning and DQN

Stabilizing Deep RL

Policy Gradients and PPO

Practical Recipe

Debugging and Reporting

Summary

RL as Learning from Interaction

Why Reinforcement Learning Is Different

Supervised learning: every example arrives with a label.

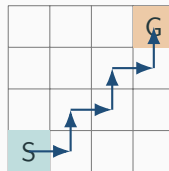
Reinforcement learning: the learner produces its own next training situation.

- The agent acts; the environment changes.
- Feedback may arrive next step, many steps later, or only at episode end.
- Behavior changes the data the agent sees.

The applied question: did the trained policy beat a baseline under a fixed evaluation rule?

Running Example: 4×4 Gridworld

- Start in lower-left, goal in upper-right.
- Actions: up, down, left, right.
- Each ordinary move: reward -0.01 .
- Reach goal: reward $+1$.
- Bumping a wall: stay in place.



A random agent wanders. A useful agent finds the goal quickly.

Defining Reinforcement Learning

Reinforcement learning

Learning behavior from interaction. At each time t the agent observes, chooses action a_t , receives reward r_{t+1} , and observes again. The objective is to make accumulated reward high.

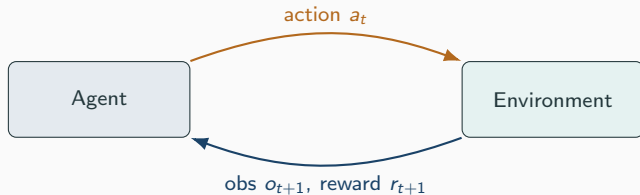
Common across very different settings:

- Gridworld coordinates, CartPole physics.
- Atari image stacks, robot sensors.
- Language model: prompt is observation, generated token is action.

Common thread: the action changes what evidence arrives next.

Problem Setting and Formalism

The Agent–Environment Loop



State s_t the underlying situation

Observation o_t what the agent actually sees

Action a_t the agent's choice

Reward $r_{t+1} \in \mathbb{R}$ scalar feedback

Trajectory, Episode, Return, Discount

Trajectory: $\tau = (o_0, a_0, r_1, o_1, a_1, r_2, \dots, o_T)$.

Discounted return from time t :

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots, \quad 0 \leq \gamma \leq 1.$$

Example: rewards 1, 0, 2 with $\gamma = 0.9$:

$$G_0 = 1 + 0.9 \cdot 0 + 0.81 \cdot 2 = 2.62.$$

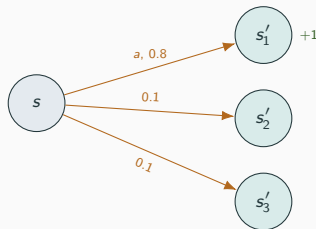
Discounting makes a shorter path to the same goal more valuable.

Markov Decision Process

MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$

- states $\mathcal{S}$, actions $\mathcal{A}$
- $P(s' | s, a)$ transition prob.
- $R(s, a, s')$ expected reward
- γ discount

Markov: next state depends only on (s, a) .



Policies, Baselines, Evaluation

Policy

A rule for choosing actions. Deterministic: $a = \pi(o)$. Stochastic: $\pi(a \mid o)$, a distribution.

Why stochastic policies?

- Useful for exploration during learning.
- Natural when several actions are plausible.
- Required by sampling-based algorithms (PPO).

Example: from start cell, $\pi(\text{right}) = 0.70$, $\pi(\text{up}) = 0.20$, others 0.05.

Training vs Evaluation

These answer different questions:

- **Training reward** measures the learning process (with ϵ -greedy, entropy bonuses, noise).
- **Evaluation return** measures the policy you actually claim.

Mean evaluation return over 20 episodes: $3700/20 = 185$.

Success rate: $43/50 = 86\%$.

Always report:

- environment name + version, wrappers
- seed policy, number of evaluation episodes
- whether actions were deterministic during evaluation

Rising training curve $\neq$ rising evaluation return.

Baselines Before Algorithms

Do not call a trained policy good until it beats a simple reference.

Random policy verifies the environment runs and shapes are right.

Heuristic policy move horizontally then vertically toward the goal.

Trained policy only meaningful relative to the above.

In Atari, a random policy scores poorly — yet it confirms wrappers, observations, and reward logging before any training is launched.

Values, Q-Functions, Credit Assignment

Value and Q-Functions

Value $V^\pi(s)$

Expected return starting in s and following π : $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$.

Q-function $Q^\pi(s, a)$

Expected return after taking a in s and then following π : $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$.

Q compares actions in the same situation.

Example: $Q^\pi(s, \text{right}) = 0.82$, $Q^\pi(s, \text{up}) = 0.61 \Rightarrow \text{right}$ has larger expected return.

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Whether a is better or worse than π 's average choice in s .

$V^\pi(s) = 0.70$, $Q^\pi(s, \text{right}) = 0.82$ gives $A = 0.12$ (good).

$Q^\pi(s, \text{left}) = 0.50$ gives $A = -0.20$ (worse than average).

Positive advantage pushes a sampled action up; negative pushes it down. Advantage is the signal at the heart of policy gradients and PPO.

Temporal-Difference Target

One-step TD target:

$$y = r + \gamma(1 - d)V(s'), \quad \text{TD error} = y - V(s).$$

$d = 1$ for terminal transitions; $d = 0$ otherwise.

Example: $r = 1$, $\gamma = 0.9$, nonterminal, $V(s') = 5$.

$y = 1 + 0.9 \cdot 5 = 5.5$. If $V(s) = 4.8$, TD error = 0.7.

TD methods correct V without waiting for the episode to finish — bootstrap from one step plus an estimate.

Q-Learning and DQN

Q-Learning Update

Q-learning target:

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a').$$

Read it as: *one observed reward plus one discounted guess about the best next action.*

Example: $r = 1$, $\gamma = 0.9$, $Q(s', a_1) = 2.0$, $Q(s', a_2) = 1.5$, nonterminal.

$$y = 1 + 0.9 \cdot \max(2.0, 1.5) = 2.8.$$

Current $Q(s, a) = 2.1 \Rightarrow$ error 0.7. Train the network's prediction for (s, a) toward 2.8.

DQN

Neural network $Q_\theta(o, a)$ predicting action values from observations. Loss on one transition:

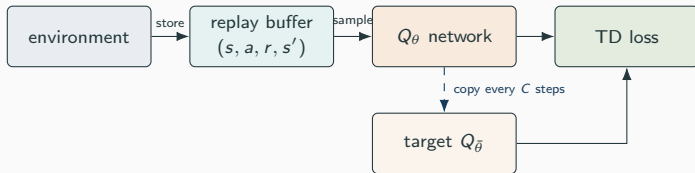
$$\ell(\theta) = (y - Q_\theta(s, a))^2.$$

Epsilon-greedy exploration: with probability $1 - \epsilon$ pick $\arg \max_a Q_\theta$; with probability ϵ pick a random action.

DQN is a package, not just a network:

- replay buffer
- target network
- observation preprocessing
- evaluation protocol

Replay Buffer and Target Network



Replay buffer: reuse and decorrelation. Target net: stable bootstrap target.

Stabilizing Deep RL

Why Deep RL Is Unstable

- The agent changes its own data distribution.
- Targets depend on the same network being trained.
- Rewards may be sparse, delayed, clipped, or noisy.
- One run may overlearn from a lucky short path.

Misleadingly high Q estimates $\Rightarrow$ agent repeats a bad action.

The data distribution and the training target both move during learning.

Double DQN, Dueling, Prioritized Replay

Double DQN: separate selection from evaluation:

$$y = r + \gamma Q_{\text{target}} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right).$$

Reduces optimistic bias from taking max over noisy estimates.

Dueling DQN: split Q into a state value stream + action advantage stream. Helpful when many actions have similar consequences.

Prioritized replay: sample transitions with larger TD error more often. Use importance-sampling weights to correct for the sampling bias.

Treat these as tools for diagnosed problems, not a checklist.

Policy Gradients and PPO

Directly adjust the policy so good actions become more likely.

Sample-based gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_t[\nabla_{\theta} \log \pi_{\theta}(a_t \mid o_t) A_t].$$

- Positive advantage $\Rightarrow$ increase log-prob of sampled action.
- Negative advantage $\Rightarrow$ decrease log-prob.
- The log appears because changing the probability of a sampled action is the direct way to change a stochastic policy.

Actor the policy that chooses actions.

Critic a value model estimating expected return or advantage.

The actor says *what to do*.

The critic says *how good the situation looks*.

If the critic is inaccurate, the actor receives poor guidance — which is why PPO logs both policy and value-function metrics.

PPO: Probability Ratio

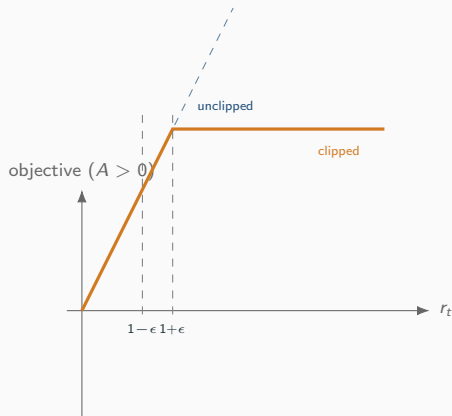
$$r_t(\theta) = \frac{\pi_{\theta}(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}.$$

Old probability 0.20, new probability 0.30 $\Rightarrow$ ratio 1.5: the new policy made the sampled action 50% more likely.

PPO clips this ratio so a single high-reward sample cannot trigger an unbounded jump.

PPO Clipped Objective

$$\mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] .$$



PPO does not forbid the policy from moving past $[1 - \epsilon, 1 + \epsilon]$ — it stops rewarding the extra

Practical Recipe

A Smoke Test Before Training

Before launching DQN or PPO:

1. Create the environment, reset it.
2. Take one action; print obs, action, reward, terminated, truncated, next obs.
3. Run a random-policy baseline for some episodes.

A failed smoke test is much cheaper than a failed training run.

Five-line check catches more bad RL experiments than another hidden layer.

What to Record per Run

Quantity	Why	What to record
Environment	defines obs, actions, rewards	name, version, wrappers, max length
Baseline	shows training improves something	random + heuristic returns
Algorithm	defines logs and signals	DQN / PPO + impl. source
Budget	fairness and cost	env steps, eval episodes, time
Speed	is the recipe usable?	wall-clock, hardware, FPS
Evidence	separates learning from noise	curves, seeds, qualitative

Lock the Evaluation Rule

Decide *before* comparing settings:

- Are evaluation actions greedy, deterministic, or sampled?
- Same number of episodes for baseline, agent, ablation.
- Multi-seed when feasible; otherwise label as **single-seed feasibility** not robust conclusion.

Ablation rule: change exactly one factor.

For DQN: replay size, target update interval, env steps, learning rate.

For PPO: clip range, parallel envs, entropy coef, env steps.

Multiple changes may improve the score, but they cannot tell you what mattered.

A Reference CartPole Run

Field	Result
Environment	CartPole-v1, max 500, success ≥ 475
Algorithm	SB3 PPO, MlpPolicy, seed 7, 4 envs, 10k steps
Random baseline	21.9 over 20 eval episodes; success 0%
Trained agent	500.0 over 20 deterministic eval; success 100%
Time / hardware	8.34 s on CPU; SB3 2.7.0, PyTorch
Ablation (clip range)	0.1 $\rightarrow$ 292.15, 0.2 $\rightarrow$ 500.0, 0.3 $\rightarrow$ 176.05

The ablation calls for humility, not a theorem about PPO clip ranges.

Debugging and Reporting

Where to Look First

RL adds a different first suspect: the interaction loop.

1. Wrappers different in train vs eval?
2. Reward sign correct?
3. Terminal flags handled?
4. Observation normalization consistent?
5. One lucky seed driving the result?

Inspect the loop before changing the architecture.

Reward hacking

The agent increases the measured reward by exploiting the reward definition rather than solving the intended task.

Examples:

- Coin-collector that never finishes the level.
- Goal-cell agent that returns to a non-terminating goal repeatedly.
- Recommender that maximizes clicks while reducing satisfaction.

The optimizer is not malicious — the reward defines the behavior that gets optimized.

Seed Sensitivity and Honest Claims

Seed sensitivity: dependence on initialization, env randomness, exploration randomness, data order.

Two seeds: 190 and 80. Average matters less than the fact that the run is unstable.

A single-seed CartPole success:

- useful as a feasibility statement
- weak evidence for a hyperparameter recommendation

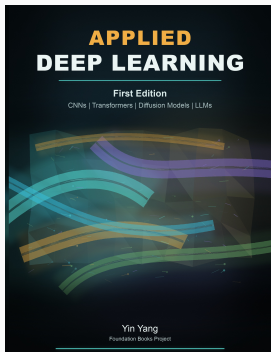
Reports should say how the agent failed: stuck at start, short-term reward loop, easy starts only, or training too slow.

Summary

Summary

- RL = behavior learned from interaction. Trajectories, returns, policies, V , Q , advantage are the central objects.
- DQN learns Q -values with a neural net; replay, target nets, Double/Dueling/Prioritized are stability tools for a moving data stream.
- PPO directly adjusts the policy with clipped probability ratios and an advantage signal — and the vocabulary carries straight into LLM post-training.
- Evidence is the deliverable: environment + wrappers, seeds, budget, fixed evaluation rule, wall-clock, and behavior next to score.
- One controlled ablation per claim.

**Do not launch an algorithm and admire a curve.
First check the reward, wrappers, evaluation rule, and a few
trajectories so the score has something honest to mean.**



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.